



Infrared PR 655

Security Review

Cantina Managed review by:
R0bert, Lead Security Researcher
Slowfi, Security Researcher

January 24, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	Deploy script sets auction to EOA	4
3.1.2	Potential control window mismatch	4
3.1.3	Revocation does not free validator	5
3.1.4	Validator validity not rechecked on claim	5
3.1.5	lastClosingPrice is not updated for expired unclaimed auctions	6
3.1.6	Auctions can be started while the system is paused	6
3.1.7	Approver does not check NFT ownership	6
3.1.8	Cooldown keyed by tokenId not validator	7
3.2	Low Risk	7
3.2.1	Duplicate receiver validation gap	7
3.2.2	Queued allocation approval gap	8
3.2.3	Active auction helpers do not account for auction expiry	8
3.2.4	Approval path does not revalidate weights against current BeraChef policy	9
3.2.5	Expiry boundary mismatch between control NFT validity and validator availability	9
3.2.6	Previous owner can cancel proposals after NFT transfer	10
3.2.7	maxAuctions cap immutable after start	10
3.3	Gas Optimization	10
3.3.1	Redundant startBlock storage write	10
3.3.2	Single-pass pending proposal scan	11
3.4	Informational	11
3.4.1	Payment token transfer assumptions	11
3.4.2	Deploy step4 requires owner	12
3.4.3	Manager must have keeper role	12
3.4.4	Manager gets broad Infrared keeper powers	12
3.4.5	Operator mismatch blocks allocations	13
3.4.6	Redundant weight validation	13
3.4.7	Unused custom errors	14
3.4.8	proposalValidityDuration not bounded	14
3.4.9	Expired proposals can block new proposals until manually cancelled	14

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Infrared simplifies interacting with Proof of Liquidity with liquid staking products such as iBGT and iBERA.

From Jan 19th to Jan 23rd the Cantina team conducted a review of [infrared-contracts](#) on commit hash [5f70465a](#). The team identified a total of **26** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	8	8	0
Low Risk	7	6	1
Gas Optimizations	2	1	1
Informational	9	5	4
Total	26	20	6

2.1 Scope

The security review had the following components in scope for [infrared-contracts](#) on commit hash [5f70465a](#):

```
src/periphery
├─ CuttingBoardDutchAuction.sol
├─ CuttingBoardManager.sol
├─ CuttingBoardNFT.sol
```

3 Findings

3.1 Medium Risk

3.1.1 Deploy script sets auction to EOA

Severity: Medium Risk

Context: [DeployCuttingBoardAuction.s.sol#L38](#)

Description: The deployment script constructs the NFT with the auction address set to `msg.sender`. Because the script broadcasts from the deployer key, the NFT's immutable auction is permanently the deployer EOA. The actual auction contract is deployed later and can never call mint due to the `onlyAuction` check, so claims will revert and the auction system is bricked. In addition, the manager allows the NFT's auction address to bypass ownership checks when proposing cutting boards, so the deployer EOA becomes a privileged actor that can submit proposals for any `tokenId`. The NFT contract does not prevent minting multiple tokens for the same validator, so a compromised or careless deployer key can bypass the auction mechanics entirely.

Recommendation: Remove the circular dependency by making the auction address settable once by the owner and set it to the deployed auction contract, or deploy the auction deterministically and pass that address into the NFT constructor. Add a wiring check in the auction initialization to assert that the NFT's auction address equals the auction contract address to prevent misconfiguration.

Infrared Finance: Fixed in commit [ca2e49f](#).

Cantina Managed: Fix verified. [DeployCuttingBoardAuction.s.sol](#) now pre-computes the auction proxy address and passes that predicted address into the [CuttingBoardNFT](#) constructor instead of `msg.sender`, eliminating the EOA-only auction wiring and ensuring the NFT is bound to the actual auction proxy at deployment.

3.1.2 Potential control window mismatch

Severity: Medium Risk

Context: [CuttingBoardDutchAuction.sol#L477](#), [CuttingBoardManager.sol#L290-L312](#), [CuttingBoardNFT.sol#L143-L153](#)

Description: Control rights are defined with a timestamp-based expiry in the NFT, while reward allocation activation is defined in blocks. When a proposal is approved, the manager only checks that the NFT is valid at approval time and then queues a new reward allocation with a start block equal to the current block plus the configured delay. Because the delay is block-based and can be large, the queued allocation can activate after the NFT has already expired in wall-clock time. This allows a controller to influence allocations beyond the intended control horizon and can harm the next auction winner or the protocol's intended time-bounded control model:

```
expiryTimestamp = block.timestamp + allocationDuration;
proposal.startBlock = uint64(block.number)
    + rewardAllocationBlockDelay
    + 1;
```

Example exploitation:

1. Alice wins an auction that grants 14 days of control (expiry at `T0 + 14 days`) and wants to maximize rewards to her preferred vaults.
2. The chain has a `rewardAllocationBlockDelay` equivalent to ~2 days of wall-clock time.
3. Alice waits until the last hour of her control window (e.g., at `T0 + 13 days 23 hours`) and submits a proposal.
4. The keeper approves the proposal immediately. The manager checks NFT validity at approval time (still valid) and queues the allocation at `block.number + delay`.
5. Two days later (after `T0 + 15 days 23 hours`), the allocation activates. Alice's allocation now controls emissions after her NFT has expired, reducing or fully overriding the next winner's allocation during their early window.

Result: Alice extends influence beyond her paid control period without paying for the extra time, harming fairness and potentially the next auction winner's expected rewards.

Recommendation: Align the control window with the block-based activation model. A straightforward fix is to add an approval-time guard that requires the NFT expiry to be sufficiently far in the future to cover the reward allocation delay and a safety buffer. Alternatively, move to a block-based expiry (or store both) and enforce that queued allocations only activate while the NFT is still valid.

Infrared Finance: Fixed in commit [8f4a0ce](#).

Cantina Managed: Fix verified. `approveCuttingBoard` now fetches `rewardAllocationBlockDelay` and enforces that `block.timestamp + 2 * beraBlockDelay <= expiry`, reverting if the NFT would expire before (or too soon after) the delayed activation, preventing approvals that would activate outside the control window.

3.1.3 Revocation does not free validator

Severity: Medium Risk

Context: [CuttingBoardDutchAuction.sol#L855-L884](#), [CuttingBoardManager.sol#L508-L514](#)

Description: Revoking an NFT only toggles its active flag in the NFT contract, but validator availability for new auctions is tracked separately by a timestamp set at claim time. The auction availability check considers the validator unavailable until the stored expiry time passes, regardless of whether the NFT has been invalidated. As a result, an emergency revocation immediately disables the holder from proposing updates, but it does not release the validator for re-auction until the original allocation duration elapses. This creates a dead period with no valid control path and undermines the purpose of a revocation mechanism during emergencies or misbehavior.

```
function revokeControl(uint256 tokenId) external onlyGovernor {
    controlNFT.invalidate(tokenId);
}

function _isValidatorAvailable(bytes32 validatorHash) internal view returns (bool) {
    if (activeValidatorAuctions[validatorHash] != 0) return false;
    if (validatorControlExpiry[validatorHash] > block.timestamp) return false;
    return true;
}
```

Recommendation: Unify control state by deriving availability from the NFT validity or by tracking the controlling tokenId per validator and checking its validity during availability checks. If the timestamp model is retained, add a governor or manager callable release function that can set the validator control expiry to the current time during revocation so the validator becomes immediately available for re-auction.

Infrared Finance: Fixed in commit [6e90b95](#).

Cantina Managed: Fix verified. The auction now tracks `validatorControlTokenId` instead of a stored expiry timestamp and `_isValidatorAvailable` checks `controlNFT.isValid(tokenId)`. Because `revokeControl` invalidates the NFT, availability immediately reflects the revocation and the validator can be re-auctioned without waiting for the original expiry.

3.1.4 Validator validity not rechecked on claim

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: Validator validity is only checked when an auction is started. The claim flow then mints the NFT, transfers payment and proposes the initial cutting board without re-validating that the validator is still registered with Infrared. If governance removes a validator between auction start and claim, a user can still pay and receive an NFT, but the subsequent approval will fail when Infrared rejects the invalid validator. This leaves the buyer with unusable control rights and creates a poor economic outcome for users who claim after a validator is removed.

```
if (!infrared.isInfraredValidator(validatorPubkey)) {  
    revert InvalidValidatorPubkey();  
}
```

Recommendation: Re-check validator validity in the claim path before state updates and payment transfer and revert if the validator is no longer registered. For clearer errors, also add an approval-time guard in the manager before calling `Infrared` so failed approvals do not waste gas.

Infrared Finance: Fixed in commit [839659d](#).

Cantina Managed: Fix verified. `claimCuttingBoardControl` now re-checks `infrared.isInfraredValidator(validatorPubkey)` immediately before processing the claim, ensuring removed validators cannot be claimed and preventing users from paying for invalid control.

3.1.5 `lastClosingPrice` is not updated for expired unclaimed auctions

Severity: Medium Risk

Context: [CuttingBoardDutchAuction.sol#L330-L351](#)

Description: The contract relies on `lastClosingPrice` as an input for subsequent auction pricing, but this value is only updated when an auction is successfully claimed. If an auction expires without being claimed, `lastClosingPrice` remains unchanged.

As a result, the system continues to use a stale closing price for price discovery, even though the previous auction has effectively failed. This can cause new auctions to be initialized with misaligned pricing assumptions, potentially leading to repeated failures to clear or prolonged periods where auctions do not converge to a valid market price.

Recommendation: Consider updating `lastClosingPrice` when an auction expires without being claimed, for example by resetting it to `basePrice` on expiry. Alternatively, consider introducing a bounded administrative mechanism to update `lastClosingPrice` in such edge cases, ensuring that price discovery can recover from unclaimed auctions.

Infrared Finance: Fixed in commit [e94eb1f](#).

Cantina Managed: Fix verified. When starting a new auction, the code now detects an expired unclaimed previous auction and updates `lastClosingPrice` to the prior auction's `basePrice`, ensuring pricing advances even if a round goes unclaimed.

3.1.6 Auctions can be started while the system is paused

Severity: Medium Risk

Context: [CuttingBoardDutchAuction.sol#L322-L334](#)

Description: The function `startCuttingBoardAuction` from contract `CuttingBoardDutchAuction` can be called while the contract is in a paused state. The pause mechanism is typically intended to halt state transitions during emergencies; however, because auction initialization is not gated by a pause check, new auctions can still be started while the system is paused. These auctions may continue running and even expire silently during the pause window. As a result, governance or operators may reasonably assume that auction-related activity is fully halted, while in practice auction state continues to evolve in the background, potentially leading to missed auctions or inconsistent expectations once the system is unpaused.

Recommendation: Consider adding a `whenNotPaused` check to `startCuttingBoardAuction` so that auctions cannot be started while the system is paused.

Infrared Finance: Fixed in commit [8a6a6a4](#).

Cantina Managed: Fix verified. `startCuttingBoardAuction` now includes the `whenNotPaused` modifier, preventing new auctions from being started while the contract is paused.

3.1.7 Approval does not check NFT ownership

Severity: Medium Risk

Context: [CuttingBoardManager.sol#L271-L297](#)

Description: The function `approveCuttingBoard` from contract `CuttingBoardManager` approves and queues a pending proposal as long as it (i) exists, (ii) has not expired, and (iii) the control NFT is still valid. However, it does not verify that the party who currently owns the control NFT still authorizes the proposal being approved.

Since proposals are stored per `tokenId`, a proposal created by a previous owner can remain pending across an NFT transfer and later be approved by a keeper, even if the new owner would not have proposed those weights. While the new owner may cancel the proposal manually, the current flow allows a keeper to approve a stale proposal before the new owner notices and cancels it, resulting in a cutting board update that does not reflect the new owner's intent.

Recommendation: Consider enforcing an ownership-based authorization at approval time (e.g., require that the current NFT owner matches `proposal.proposer`, or require an explicit approval/ack from the current owner). Alternatively, consider invalidating any pending proposal on NFT transfer (e.g., via a transfer hook pattern between the NFT and the manager, or by requiring proposals to include a nonce that changes on transfer).

Infrared Finance: Fixed in commit [155d04c](#).

Cantina Managed: Fix verified. `approveCuttingBoard` now requires `proposal.proposer` to equal the current NFT owner and reverts with `InvalidProposer` if ownership has changed, preventing approval of stale proposals after transfers.

3.1.8 Cooldown keyed by tokenId not validator

Severity: Medium Risk

Context: *(No context files were provided by the reviewer)*

Description: The system has two different perspectives on update eligibility. `BeraChef` enforces queued allocations per validator pubkey and allows only one queued allocation at a time. The manager enforces a cooldown per NFT `tokenId`, not per validator. This dimension mismatch means that when a new auction mints a new `tokenId` for the same validator, the manager treats it as a fresh update context even though the validator may still have a queued allocation in `BeraChef`. The manager gate and tracking are per `tokenId` and are only set on approval:

```
if (lastUpdateBlock[tokenId] != 0
    && block.number < lastUpdateBlock[tokenId] + rewardAllocationBlockDelay)
{
    revert UpdateTooSoon();
}

lastUpdateBlock[tokenId] = block.number;
```

`BeraChef` maintains the queue per validator and rejects a second queued allocation for the same pubkey:

```
RewardAllocation storage qra = queuedRewardAllocations[valPubkey];
if (qra.startBlock > 0) {
    revert RewardAllocationAlreadyQueued();
}
```

A realistic exploitation scenario is a normal auction cycle. Winner A queues an allocation late in their control window. That queued allocation remains in `BeraChef` until the distributor activates it. A new auction then starts for the same validator and mints a new `tokenId` for winner B. Because `lastUpdateBlock` for the new `tokenId` is zero, the manager allows winner B to propose immediately and a keeper may attempt approval right away. Approval reverts because `BeraChef` still has the old queued allocation for the validator. The keeper can retry but will keep reverting until the old queue is cleared. If activation is delayed long enough, the proposal can expire and winner B loses the ability to set an initial allocation despite paying for control. This creates repeated revert spam and a practical denial of control for new winners.

The core of the issue is that the manager gates by `tokenId`, while the queueing rule is enforced by validator pubkey. A new NFT resets the manager gate even though the validator queue is still occupied.

Recommendation: Track cooldown and queue eligibility per validator pubkey rather than per `tokenId`. Store a `nextEligibleBlock` keyed by validator hash and enforce it for every proposal, including the first update of a new `tokenId`. Optionally add an on chain check against BeraChef queue state and reject approvals while a queued allocation exists so the manager fails early with a clear error instead of relying on BeraChef reverts.

Infrared Finance: Fixed in commit [10cc01f](#).

Cantina Managed: Fix verified. `approveCuttingBoard` now checks `chef.getQueuedRewardAllocation(pubkey)` and reverts `AlreadyQueue` when a queued allocation exists, preventing approvals while the validator queue is occupied and removing the practical failure mode described.

3.2 Low Risk

3.2.1 Duplicate receiver validation gap

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: BeraChef rejects reward allocations that include duplicate receivers, but the periphery validation in the auction and manager does not check for duplicates. This allows a user to claim an auction and store a proposal that later fails during keeper approval when BeraChef detects duplicate receivers. The user can cancel and re-propose, so the issue is not permanent, but it creates a realistic paid-then-failed approval path and inconsistent validation behavior.

Recommendation: Mirror BeraChef's duplicate receiver check in the manager validation (and optionally in the auction validation) so invalid allocations are rejected earlier with clear errors.

Infrared Finance: Fixed in commit [04e4e7f](#).

Cantina Managed: Fix verified. `CuttingBoardManager` now checks for duplicate receivers in `_validateWeights` via `_checkForDuplicateReceivers`, so proposals with duplicates are rejected at propose time before reaching BeraChef.

3.2.2 Queued allocation approval gap

Severity: Low Risk

Context: [CuttingBoardManager.sol#L237-L244](#), [CuttingBoardManager.sol#L305-L307](#)

Description: Berachef only allows a single queued reward allocation per validator and it only clears that queue when the distributor activates it. The manager approval flow does not check whether a queued allocation already exists, and it also uses a readiness condition based on `lastUpdateBlock` plus the delay, while Berachef enforces a stricter `startBlock` rule that must be greater than the current block plus the delay. These mismatches create two practical gaps. First, a keeper can attempt to approve a proposal while a previous allocation is still queued in Berachef, causing an immediate revert even though the manager believes the proposal is ready. Second, there is an off by one timing window where the manager considers the delay satisfied, but Berachef still rejects the queue call because the computed `startBlock` is not strictly far enough in the future. In both cases, the approval transaction fails and must be retried later, wasting keeper gas and creating intermittent approval failures until the distributor activates the prior allocation or the block height advances enough to satisfy the strict rule.

For example, if a validator already has a queued allocation waiting for activation, a new proposal can be approved by the manager without any guard, but Berachef will revert with `RewardAllocationAlreadyQueued` and the proposal will remain pending. Separately, if `lastUpdateBlock` plus the delay equals the current block, the manager will attempt approval and set `startBlock` to current block plus delay plus one, yet Berachef requires the `startBlock` to be strictly greater than current block plus delay, which can still fail depending on block timing. This produces a short lived but real approval gap during which approvals fail despite the manager's readiness check.

```
if (startBlock <= block.number + rewardAllocationBlockDelay) revert;
```

Recommendation: Have keeper automation check for a queued allocation before approving, or extend the manager to read the queued allocation state from BeraChef and block approvals when one already exists. Alternatively, adjust the manager's readiness check to align with BeraChef's strict startBlock requirement.

Infrared Finance: Fixed in commit [10cc01f](#).

Cantina Managed: Fix verified. `approveCuttingBoard` now reads `chef.getQueuedRewardAllocation(pubkey)` and reverts `AlreadyQueue` when a queued allocation exists, preventing approvals that would revert in BeraChef. It also sets `startBlock` to `block.number + beraBlockDelay + 1`, matching BeraChef's strict delay requirement.

3.2.3 Active auction helpers do not account for auction expiry

Severity: Low Risk

Context: [CuttingBoardDutchAuction.sol#L624-L635](#), [CuttingBoardDutchAuction.sol#L642-L660](#)

Description: The functions `getActiveAuction` and `isAuctionActive` determine whether an auction is active based on stored auction state, but they do not validate whether the auction has already expired. In particular, both helpers omit a check ensuring that `block.timestamp` is still lower than `startTime + auctionDuration`.

As a result, once an auction has elapsed, these functions may continue to report it as active even though it is no longer valid from a timing perspective. This can cause off-chain consumers such as bots, keepers, or UI components to believe that an expired auction is still ongoing, which may prevent keepers from starting a new auction and lead to stalled auction cycles.

Recommendation: Consider extending the logic in both `getActiveAuction` and `isAuctionActive` to also check that `block.timestamp < startTime + auctionDuration` when determining whether an auction is active.

Infrared Finance: Fixed in commit [20db4f2](#).

Cantina Managed: Fix verified. Both `isAuctionActive` and `getActiveAuction` now include the `block.timestamp < auction.startTime + auctionDuration` guard, so expired auctions are no longer reported as active.

3.2.4 Approval path does not revalidate weights against current BeraChef policy

Severity: Low Risk

Context: [CuttingBoardManager.sol#L271-L320](#)

Description: The function `approveCuttingBoard` from contract `CuttingBoardManager` queues the previously proposed `proposal.weights` without running the same validation performed at proposal time (e.g., vault whitelist and per-vault bounds enforced in `_validateWeights`). Since the validation is only applied in `proposeCuttingBoard`, changes in BeraChef policy (e.g., a vault becoming non-whitelisted, or parameter updates such as maximum weight / maximum number of weights) between proposal and approval are not accounted for during approval.

This can cause the approval transaction to revert (if the current policy rejects the weights), leaving the proposal stored and still blocking further proposals for the same `tokenId` until it is manually cancelled. Alternatively, if no downstream validation exists, outdated policy constraints can be effectively bypassed for proposals created before the policy change.

Recommendation: Consider validating `proposal.weights` during `approveCuttingBoard` using the current BeraChef parameters/whitelist before queueing the update. If re-validation is intentionally delegated to BeraChef, consider allowing proposal cleanup when approval fails (e.g., a cancellation path that unblocks `tokenId` when the proposal has become invalid under current policy) to reduce the risk of proposals becoming stuck.

Infrared Finance: Acknowledged. Good point. We are taking care of this in our keeper bot. Specifically we are checking for revert errors and automatically cancel if they are known to be permanent issues, like invalid weights. I will make a note for this case to be double checked.

Cantina Managed: Acknowledged.

3.2.5 Expiry boundary mismatch between control NFT validity and validator availability

Severity: Low Risk

Context: [CuttingBoardDutchAuction.sol#L878-L881](#), [CuttingBoardNFT.sol#L151-L152](#)

Description: The function `isValid` from contract `CuttingBoardNFT` treats a control NFT as valid while `block.timestamp <= rights.expiryTimestamp`. In contrast, the function `_isValidatorAvailable` from contract `CuttingBoardDutchAuction` treats the control period as expired unless `expiry > block.timestamp`, which makes the validator available for a new auction when `expiry <= block.timestamp`.

At the exact boundary where `block.timestamp == expiryTimestamp`, the NFT is still considered valid, while the validator is also considered available for re-auction. This creates a one-timestamp overlap where control rights are still reported as active but the validator can be auctioned again, which can lead to inconsistent behavior across contracts and consumer tooling (e.g., UI/keepers observing "valid control" while a new auction can already be started).

Recommendation: Consider aligning the boundary condition between the NFT validity check and the validator availability check (e.g., use either strict `<` semantics on both sides or inclusive `<=` semantics on both sides) so that there is no timestamp where both "still valid" and "available for auction" are simultaneously true.

Infrared Finance: Fixed in commit [6e90b95](#)

Cantina Managed: Fix verified. Validator availability now derives directly from `controlNFT.isValid(tokenId)`, so the auction's availability check uses the exact same `<= expiryTimestamp` boundary as the NFT validity check, eliminating the overlap at the expiry timestamp.

3.2.6 Previous owner can cancel proposals after NFT transfer

Severity: Low Risk

Context: [CuttingBoardManager.sol#L326-L343](#)

Description: The function `cancelProposal` from contract `CuttingBoardManager` allows a proposal to be cancelled by the original `proposal.proposer` even after ownership of the control NFT has been transferred. The proposer address is stored at proposal creation time and remains authorized to cancel the proposal regardless of subsequent NFT ownership changes.

As a result, when a control NFT is sold with an active proposal, the previous owner (as proposer) retains the ability to cancel that proposal after the transfer. This enables the seller to unilaterally interfere with the buyer's pending update by cancelling it post-sale, even though the seller no longer controls the NFT or the validator. This behavior is surprising from an ownership perspective and can lead to disputes or unintended interference during ownership transitions.

Recommendation: Consider restricting proposal cancellation to the current NFT owner and/or governance, rather than the original proposer. Alternatively, consider clearing proposer authority or automatically deleting any pending proposal when the NFT is transferred, so that proposal control always follows current ownership.

Infrared Finance: Fixed in commit [f3d78b0](#)

Cantina Managed: Fix verified. `cancelProposal` no longer allows the original proposer to cancel, only the keeper, current NFT owner, or governance can cancel, so prior owners cannot interfere after a transfer.

3.2.7 maxAuctions cap immutable after start

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: The auction system enforces a maximum number of auctions and blocks new auctions once the limit is reached. There is a setter for `maxAuctions`, but it is only callable before any auctions have started. After the first auction is created, the cap becomes immutable. This makes `maxAuctions` a one time configuration decision at deployment time. If it is set too low, the system will permanently stop

starting new auctions once the cap is reached, and changing the value would require an upgrade rather than a normal governance action.

The behavior is enforced by the setter guard and the auction start guard:

```
function setMaxAuctions(uint256 _maxAuctions) external onlyGovernor {
    if (auctions.length > 0) revert AuctionsAlreadyStarted();
    if (_maxAuctions == 0) revert InvalidMaxAuctions();
    maxAuctions = _maxAuctions;
}

function startCuttingBoardAuction(bytes calldata pubkey) external onlyKeeper {
    if (auctions.length >= maxAuctions) revert AllAuctionsCompleted();
    // start auction
}
```

This is not a code exploit, but it is an operational hazard if the intended product behavior is continuous or long term recurring auctions. Impact: once the cap is hit, auctions stop until a contract upgrade or replacement.

Recommendation: Set `maxAuctions` to a very high value at deployment if ongoing auctions are expected, or remove the post start restriction so governance can raise the cap later. If the cap is intentionally finite, document it explicitly as a product constraint and add monitoring to alert when the cap is approaching.

Infrared Finance: Fixed in commit [6ca29b7](#).

Cantina Managed: Fix verified. `setMaxAuctions` no longer reverts once auctions have started, so governance can raise the cap post-launch.

3.3 Gas Optimization

3.3.1 Redundant startBlock storage write

Severity: Gas Optimization

Context: [CuttingBoardManager.sol#L305-L307](#), [CuttingBoardManager.sol#L318-L319](#)

Description: `approveCuttingBoard` computes the start block for the queued allocation by writing `proposal.startBlock` and then immediately deletes the proposal struct at the end of the call. The stored `startBlock` value is never read from storage, because the function uses it only in the same call to queue the cutting board and emit the approval event. This makes the storage write unnecessary and costs extra gas for every approval without providing any persistence or safety.

Recommendation: Compute the start block as a local variable and use it for `queueNewCuttingBoard` and the event, then delete the proposal without writing `proposal.startBlock`.

Infrared Finance: Fixed in commit [fe7bd73](#).

Cantina Managed: Fix verified. `approveCuttingBoard` now uses a local `startBlock` variable and no longer writes `proposal.startBlock` before deleting the proposal.

3.3.2 Single-pass pending proposal scan

Severity: Gas Optimization

Context: [CuttingBoardManager.sol#L358-L392](#)

Description: `getPendingProposalTokenIds` scans the entire token range twice, calling the pending-proposal check in each pass. The check also calls into the NFT contract to verify validity, so the doubled pass repeats the same external view calls and bookkeeping.

Recommendation: Use a single pass that writes into a preallocated array sized to `totalSupply` and then shrink the length with assembly, or maintain a separate index of pending proposals to avoid full scans.

Infrared Finance: Acknowledged.

Cantina Managed: Acknowledged.

3.4 Informational

3.4.1 Payment token transfer assumptions

Severity: Informational

Context: [CuttingBoardDutchAuction.sol#L498-L499](#)

Description: The claim path transfers the payment token to the treasury without validating the amount actually received. This assumes the payment token transfers the exact requested amount and does not apply transfer fees, rebases or restrictions. If a fee-on-transfer or deflationary token is configured, the winner can receive full control rights while the treasury receives less than the recorded price and the stored closing price no longer reflects value actually paid. If the token can blocklist or otherwise restrict transfers, claims can revert and leave an auction unclaimable, which can cascade into a system-wide halt because new auctions cannot start until the previous one is claimed.

```
paymentToken.safeTransferFrom(msg.sender, treasury, currentPrice);
```

Example:

1. Governance mistakenly configures the payment token to a fee-on-transfer ERC20 that burns 5% on every transfer.
2. Alice claims an auction at a current price of 1000 tokens and receives the control NFT.
3. The transfer executes, but the treasury only receives 950 tokens due to the fee.
4. The contract records `lastClosingPrice` as 1000, so subsequent auctions are priced off an inflated closing price, while the treasury is underfunded relative to the auctioned control rights.

Alternative liveness failure scenario:

1. The payment token implements a blocklist and the treasury address is later blocked.
2. Any claim attempt reverts because `safeTransferFrom` fails.
3. The active auction can never be claimed, and new auctions cannot be started due to the sequential-auction constraint.

Result: Either the protocol loses value on every auction (fee-on-transfer) or a single misconfiguration/blacklist event can halt the auction system entirely.

Recommendation: Either restrict the payment token to a plain ERC20 by policy and document the requirement, or enforce it on-chain by checking the treasury balance delta and requiring it to equal the expected price. If transfer restrictions are possible, consider adding an emergency bypass or force-close path so a single failed claim cannot stall all future auctions.

Infrared Finance: Acknowledged. Auction is designed around payout token being IR, which is a [vanilla token](#) We will make a note in contract comments.

Cantina Managed: Acknowledged.

3.4.2 Deploy step4 requires owner

Severity: Informational

Context: [DeployCuttingBoardAuction.s.sol#L70-L74](#)

Description: The deployment script calls `nft.setManager` during Step 4, but `setManager` is restricted to the NFT owner. The NFT owner is set to `OWNER_ADDRESS` in the constructor, while the script broadcasts from `PRIVATE_KEY`. If those are not the same address, Step 4 will revert and the manager address will not be set. This is a deployment footgun that can leave the system partially configured until the owner executes the action separately.

Recommendation: Either remove Step 4 from the script and treat it as an explicit post-deploy owner action, or ensure the script is executed by the owner. A temporary owner pattern followed by ownership transfer can also avoid this failure.

Infrared Finance: Fixed in commit [ca2e49f](#).

Cantina Managed: Fix verified. The deploy script no longer calls `nft.setManager`. Instead it pre-computes the manager proxy address and passes it into the `CuttingBoardNFT` constructor, avoiding the owner-only call that could revert under a non-owner deployer.

3.4.3 Manager must have keeper role

Severity: Informational

Context: [DeployCuttingBoardAuction.s.sol#L95-L97](#)

Description: Cutting board approvals call into `Infrared`, which restricts queueing to accounts with the keeper role. If the manager contract is not granted `KEEPER_ROLE` on `Infrared`, `approveCuttingBoard` will revert and no proposals can be finalized. The deployment script only prints this step and does not enforce or verify it, so it is easy to miss and results in a non-functional approval flow.

Recommendation: Treat the keeper role grant as a required post-deploy step and verify it before enabling auctions. Consider adding a deployment-time check or an explicit guard in the manager that detects missing role permissions and reverts with a clear error.

Infrared Finance: Acknowledged. We added this step to the deployment checklist.

Cantina Managed: Acknowledged.

3.4.4 Manager gets broad Infrared keeper powers

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The cutting board flow requires granting the `Infrared KEEPER_ROLE` to the `CuttingBoardManager` so it can call `queueNewCuttingBoard`. That role is global and authorizes multiple operational functions beyond cutting board queueing, so the Manager address inherits much broader authority than its stated purpose. This is an architectural least-privilege violation rather than a direct exploit, but it becomes material because the Manager is UUPS upgradeable and its upgrade authority is controlled by its own governance role. If Manager governance is weaker, misconfigured or compromised, the attacker can upgrade Manager to call any `onlyKeeper` function on `Infrared` even though the original Manager code does not expose such calls. This turns a scoped integration requirement into a systemic privilege escalation risk. The impact is governance and economic integrity: a single compromised Manager upgrade can alter fee settings, toggle auction base behavior, or manipulate boost queues using `Infrared`'s keeper permissions, which can degrade protocol economics or redirect rewards.

A realistic example is a compromised Manager governance key that deploys an upgraded Manager implementation with a public function that calls `Infrared.queueBoosts` and `Infrared.cancelBoosts` to reallocate boosts toward attacker-controlled validators. Because the Manager already holds `Infrared KEEPER_ROLE`, those calls succeed even though no `Infrared` governance action was approved. This does not require changing `Infrared` or compromising its governance, yet it can shift emissions and rewards in a way that is inconsistent with the intended trust boundaries.

Recommendation: Introduce a dedicated role in `Infrared` that is only authorized to call `queueNewCuttingBoard`, and grant that limited role to the Manager instead of `KEEPER_ROLE`. If changing core roles is not feasible, add a strict allowlist on `queueNewCuttingBoard` so only the Manager and approved EOAs can call it, and keep `KEEPER_ROLE` reserved for broader operational functions. Ensure that Manager and `Infrared` governance are aligned and equally secured if broad keeper privileges must be granted.

Infrared Finance: Acknowledged. Governance is our protocol multisig. In the unlikely event it is compromised, this edge case would be the least of our issues.

Cantina Managed: Acknowledged.

3.4.5 Operator mismatch blocks allocations

Severity: Informational

Context: *(No context files were provided by the reviewer)*

Description: The auction system assumes that `Infrared` is the validator operator recognized by `BeraChef` for every auctioned pubkey. `BeraChef` enforces this at queue time by requiring `msg.sender` to equal the `BeaconDeposit` operator for the pubkey, but the auction eligibility check only verifies that the pubkey is present in `Infrared`'s validator set. This creates an external invariant that is not enforced in PR655: a validator can be added to `Infrared` and be auctioned even when `BeaconDeposit` still lists a different operator or no operator at all, causing approval to revert and the queued allocation to never activate. The failure mode is a functional denial of service for that validator's auction flow rather than a direct exploit, and it is easy to trigger if validator onboarding or operator changes are done out of sequence.

A realistic scenario is governance adding a new validator to `Infrared` before its first deposit is executed. `BeaconDeposit` still reports operator as the zero address, so any attempt to queue a cutting board reverts even though the auction appears valid and claims can succeed. Another realistic case is a validator that was originally deposited under a different operator, then later added to `Infrared` without performing the operator change flow. In both cases, approvals will consistently fail and the NFT holder cannot activate allocations for that validator.

```
function queueNewRewardAllocation(...) external onlyOperator(valPubkey)

modifier onlyOperator(bytes calldata valPubkey) {
    if (msg.sender != beaconDepositContract.getOperator(valPubkey)) revert;
}
```

Recommendation: Enforce the operator invariant when validators are added or before auctions start by checking that `BeaconDeposit.getOperator(pubkey)` equals the `Infrared` contract address. If enforcing on-chain is not feasible, add a deployment and operational checklist that asserts operator alignment for every auctioned validator and block auctions for mismatched pubkeys.

Infrared Finance: Acknowledged. We are currently manually selecting our top 2 validators only for auction, which are fully staked and long since set operator.

Cantina Managed: Acknowledged.

3.4.6 Redundant weight validation

Severity: Informational

Context: `CuttingBoardDutchAuction.sol#L434-L435`, `CuttingBoardManager.sol#L246-L247`

Description: The auction claim path performs partial weight validation while the manager performs the full validation. Because a claim always calls into the manager to propose the initial cutting board, invalid weights will still be rejected there. The auction-side checks therefore duplicate work and apply a different rule set, which can cause confusion during maintenance and adds extra gas. This is not a security issue, but it is a polish and consistency concern.

Recommendation: Either remove the auction-side weight validation and rely on the manager, or align the checks so both paths enforce the same rules.

Infrared Finance: Fixed in commit `97a48d4`.

Cantina Managed: Fix verified. The auction-side `_validateWeights` call and helper were removed from `claimCuttingBoardControl`, so weight validation is now handled solely by `CuttingBoardManager` during `proposeCuttingBoard`, eliminating redundant checks.

3.4.7 Unused custom errors

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Several custom errors declared in the auction contract are never referenced. This is not a security issue, but it adds noise and makes it harder to reason about which error conditions are actually possible. The unused errors are:

```
InvalidPortion
InvalidVaultAddress
ContractPaused
VaultNotWhitelisted
```

InvalidKeeper
Unauthorized

Recommendation: Remove unused custom errors or add the missing checks that would use them so the contract surface matches the implemented behavior.

Infrared Finance: Fixed in commit [19445cb](#)

Cantina Managed: Fix verified. The listed unused custom errors were removed from `CuttingBoardDutchAuction.sol`.

3.4.8 `proposalValidityDuration` not bounded

Severity: Informational

Context: `CuttingBoardManager.sol#L495-L505`

Description: The function `setProposalValidityDuration` from contract `CuttingBoardManager` allows governance to update `proposalValidityDuration` without enforcing any bounds on the input value. This value is later used to compute proposal expiry timestamps and to determine whether proposals can be approved or considered valid.

If `proposalValidityDuration` is set to 0, newly created proposals will be immediately expired, making approvals impossible in practice and effectively freezing updates unless governance intervenes again. Conversely, setting an excessively large value can result in proposals remaining valid for an unreasonably long time, increasing the likelihood of stale proposals interacting poorly with later policy changes and prolonging the blocking behavior caused by `proposals[tokenId].exists`.

While this is a governance-controlled parameter, the lack of validation makes it easy to misconfigure the system into a broken or degraded liveness state through a single erroneous update.

Recommendation: Consider enforcing sane minimum and maximum bounds in `setProposalValidityDuration` (e.g., a non-zero lower bound and a reasonable upper bound) to reduce the risk of accidental misconfiguration that can make proposals unapprovable or excessively long-lived.

Infrared Finance: Fixed in commit [909578b](#)

Cantina Managed: Fix verified. `setProposalValidityDuration` now reverts on zero duration via `InvalidDuration`, preventing the immediate-expiry misconfiguration.

3.4.9 Expired proposals can block new proposals until manually cancelled

Severity: Informational

Context: `CuttingBoardManager.sol#L475-L489`

Description: The function `proposeCuttingBoard` from contract `CuttingBoardManager` reverts when `proposals[tokenId].exists` is set, regardless of whether the stored proposal has already expired. Similarly, the function `canProposeNow` from contract `CuttingBoardManager` returns false whenever `proposals[tokenId].exists` is set, without considering proposal expiry.

While the function `approveCuttingBoard` correctly rejects expired proposals via a timestamp check, there is no mechanism to automatically clear expired proposals from storage. As a result, an expired proposal can persist indefinitely and prevent new proposals for the same `tokenId` unless a privileged party calls `cancelProposal`.

This creates a liveness dependency on manual intervention (proposer/NFT owner/governance) to unblock updates, and if those parties cannot act promptly, cutting board updates for that validator can remain frozen until governance intervenes.

Recommendation: Consider adding a permissionless `clearExpiredProposal(tokenId)` function that deletes `proposals[tokenId]` when it is expired. Alternatively, consider auto-clearing expired proposals inside `proposeCuttingBoard` and/or `canProposeNow` (e.g., if `isProposalExpired(tokenId)` is true, treat the proposal as non-existent and/or delete it).

Infrared Finance: Fixed in commit [53842aa](#)

Cantina Managed: Fix verified. `proposeCuttingBoard` now auto-clears expired proposals before rejecting new ones, and `canProposeNow` treats expired proposals as non-blocking, removing the indefinite blockage from stale proposals.

DRAFT